Report of Assigment 2 - Big Machine Learning

Kamil Pilkiewicz

January 2026

1 Issues

I encountered several issues that need to be discussed:

- The project required running multiple jobs, while both the source scripts and the sbatch scripts were modified alternately at each stage of the task. As a result, earlier versions of the sbatch scripts were not always compatible with later versions of the source code (`main.py`). For example, in Step 2 I introduced `ArgumentParser`. Initially, I defined it without default values, which required explicitly passing all arguments. Later, I added default values to simplify execution.

- I used `wandb` for logging. Initially, I was unable to use it to collect plots demonstrating multi-GPU training. Later, I found a suggestion to change the environment variable `TMPDIR` to a local directory. After setting `export TMPDIR=$SCRATCH/tmp`, `wandb` began displaying the `System` section, which partially reports GPU utilization. Only the main process logs metrics to `wandb`.

I have attached the original versions of the sbatch scripts, each of which worked with a specific version of the source files. Every sbatch script indicates which version of `main.py` was used. All source file versions are attached in the `src_files` directory. Note that `main.py` in the main folder is equivalent to `main7_distributed_sampler.py` in the `src_files` directory, except for minor differences in comments.

2 Explanation of implementation and calculation of training steps

- **Step 2:** I added default values to `ArgumentParser`, but only for the hyperparameters introduced in Step 7.

- **Step 3:** I used `wandb` as a monitoring platform. Initially, I attempted to manually log GPU-related information. Later, I modified the sbatch

script to include:

```
export TMPDIR=$SCRATCH/tmp
```

(the directory was created beforehand). This enabled `wandb` to display the `System` section, which includes hardware utilization information. Only the main node logs data to `wandb`.

- **Step 4b:** I configured:
  - `sharding_strategy` as `ShardingStrategy.FULL_SHARD`,
  - `auto_wrap_policy` as:

    ```
    partial(transformer_auto_wrap_policy, transformer_layer_cls={Block})
    ```

  - `mp_policy` as `MixedPrecision` with precision set to `torch.bfloat16`.
- **Step 4d:** In an earlier version, I distributed the dataset using:

  ```
  hf_dataset = hf_dataset.shard(num_shards=world_size, index=rank)
  ```

  after converting it to an iterable dataset. In the final version, I replaced this approach with `DistributedSampler`, as implemented in final version `main7_distributed_sampler.py` (compare with `main7.py`).
- **Step 5:** I used `LambdaLR` with a custom function implementing the WSD learning rate schedule. The decay phase was implemented as linear decay.
- **Step 6:** I replaced the `feed_forward_layer` in `Block` with a custom `SwiGLU` class. The SwiGLU implementation uses the Swish activation function with $\beta = 1$. Both biased and bias-free variants are discussed in the paper; although the authors recommend omitting bias, I retained it to remain consistent with the original `FeedForward` implementation. I also reduced `hidden_dim` to $\frac{2}{3}$ of its original size, as proposed in the paper, in order to maintain a comparable number of parameters.
- **Step 7:** To estimate the optimal number of training steps, I first computed the number of model parameters using a custom function `count_fsdp_params`. I evaluated the parameter count both before and after applying FSDP. The correct number of parameters (before FSDP wrapping) is 30,004,736, i.e., approximately $N = 3 \cdot 10^7$.

  According to the scaling rule, we require $D = 20N = 6 \cdot 10^8$ training tokens. The batch size is 256 and the sequence length is 256, so one training step processes:

  $$T = 256 \times 256 = 2^{16} \approx 65{,}536$$

  tokens.

  Therefore, the required number of training steps is:

  $$\frac{D}{T} \approx \frac{6 \cdot 10^8}{65{,}536} \approx 9{,}200.$$

3 Plots

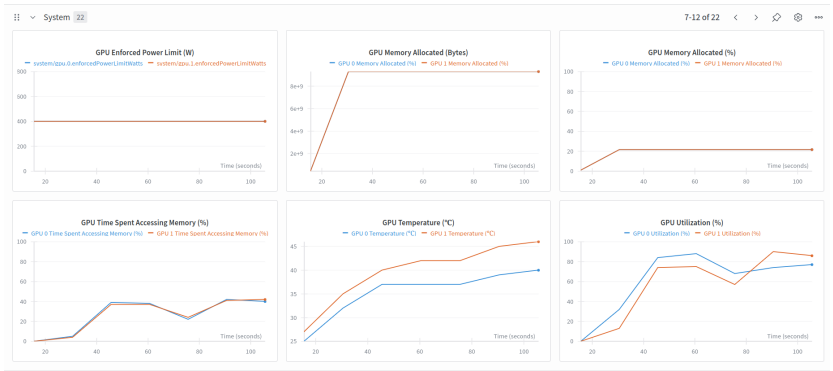

Figure 1: For 2gpus: Proof of multigpu

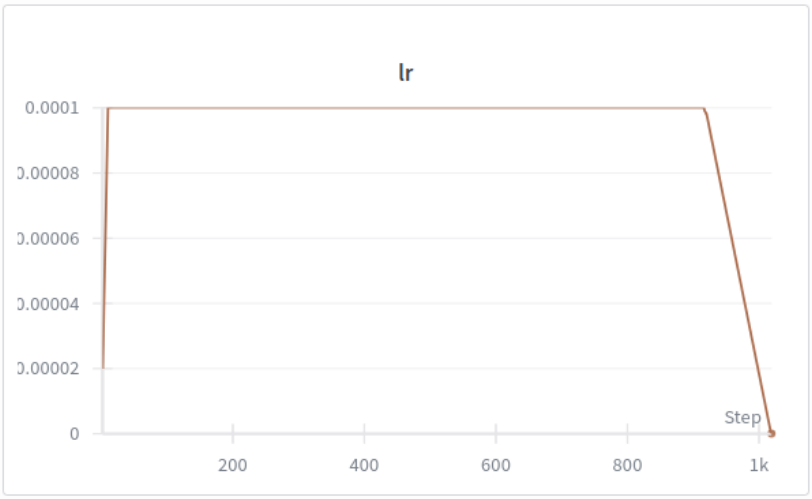

Figure 2: For 2gpus: Proof of proper WSD learning rate schedule

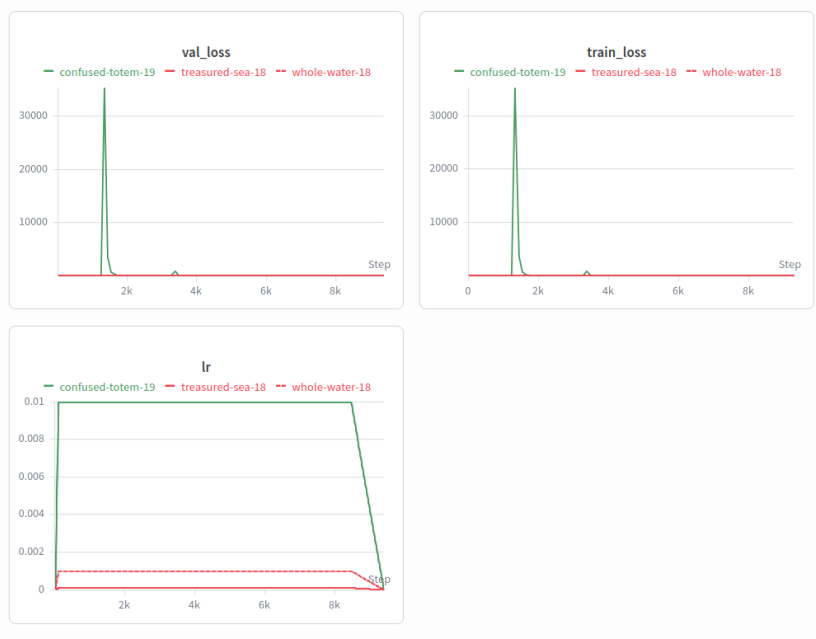

Figure 3: For grid search: Learning rate, validation and training loss for all 3 learning rates. We can see the worst one is for 0.01, as it is very unstable.

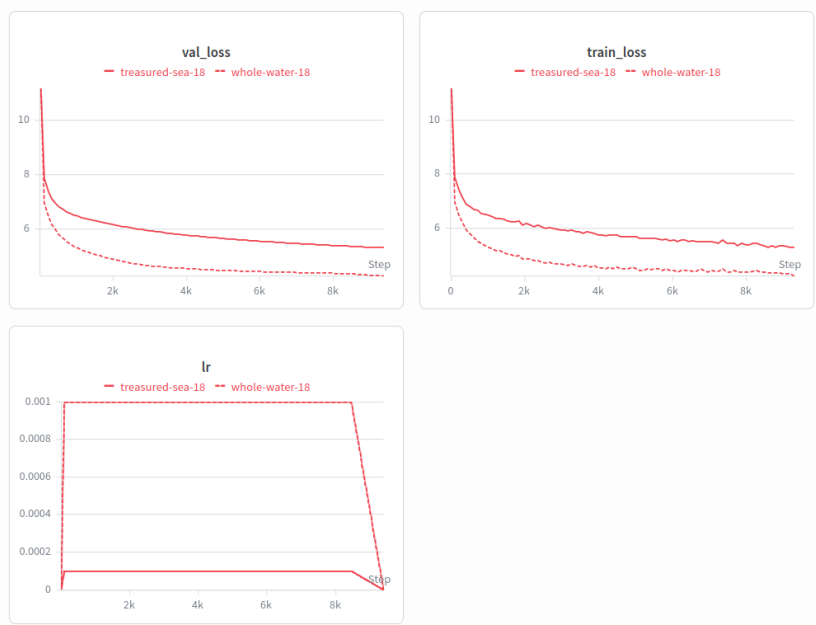

Figure 4: For grid search: Learning rate, validation and training loss. We can see that 0.001 learning rate is much faster than 0.0001 and stable. As a result we know the best learning rate is 0.001.